Ders 4 Çalışma Özeti

Ders 4 Çalışma Özeti

Genel Konular

- Proje duyuruları: grup bildirimi, proje konusu seçimi ve projede istenecek test belgeleri.
- Geçen haftanın tekrarı: yazılım incelemesi (software inspection), Fagan incelemesi, kod okuma ve V model.
- Test süreci adımları: birim testi (unit test), ara birim testi (interface test), bütünleme/bütünlük testi (integration test), regresyon testi (regression test), sistem testi (system test), onaylama testi (validation test), alfa ve beta testi, kabul testi (acceptance test).
- Test tipleri: saydam kutu testi (white box / glass box testing), kara kutu testi (black box testing) ve gri kutu testi (gray box testing).
- Saydam kutu yöntemleri: temel yol testi (basis path testing) ve döngü testi (loop testing).
- Kara kutu yöntemleri: eş değerli bölümleme (equivalence partitioning), sınır değer analizi (boundary value analysis), keşif testi (exploratory testing).
- Hata giderme (debugging), hata kaynakları ve hata düzeyleri.

Hocanın Özellikle Vurguladığı Kısımlar

- Test süreci adımları ile test tipleri farklı kavramlardır.
  - Test süreci adımları testin sırasını verir: birim testi, bütünlük testi, sistem testi, kullanıcı testi.
  - Test tipleri her adımda hangi bakış açısıyla test yapılacağını verir: saydam kutu, kara kutu, gri kutu.
- Birim testini test grubu değil, kodu yazan kişi yapar.
- Tek başına doğru çalışan modüller birleştiğinde hatalı çalışabilir; bütünlük testinin nedeni budur.
- Sistem testinde mutlaka bir araç (tool) kullanmak gerekir; çok sayıda kullanıcıyı aynı anda gerçek kişilerle denemek çok zordur.
- Saydam kutu ve kara kutu testleri birbirinin alternatifi değildir; birbirini tamamlar ve farklı karakterde hatalar bulur.
- Test verisini rastgele seçmeyin. Girdi uzayını sınıflara bölün ve sınırlara bakın.
- Temel yol testi bir saydam kutu test tipidir ve birim testi içinde uygulanır. Temel yol testi ile birim testini karıştırmayın.
- McCabe ölçüsünün (cyclomatic complexity) hesabı yazılım ölçümü konusunda anlatılacaktır.

Kısa Tekrar Notları

- İnceleme statik geçerleme ve doğrulamadır; test dinamik geçerleme ve doğrulamadır.
- İnceleme sonucu üç kararla biter: değişiklik yapmadan kabul, küçük hatalar belli sürede düzeltilmek koşuluyla kabul, ret ve düzeltme sonrası yeniden inceleme.
- Test senaryosu (test case): belirli bir program yolunu işletmek ya da bir gereksinime uygunluğu onaylamak için düzenlenen test verisi grubu.
- Test sürücü (test driver): test edilen birimi çağıran yapay programdır. Kukla modül (stub): test edilen birimin çağırdığı, henüz test edilmemiş modülün yerine geçen yapay alt programdır.

- Bütünlük testi iki biçimde yapılır: bütün olarak (big bang) ya da artırmalı (incremental). Artırmalı yol yukarıdan aşağı, aşağıdan yukarı veya ikisinin birleşimi olan sandviç testi olabilir.
- Regresyon testi: test edilmiş bir programda değişiklik veya ekleme yapılınca bütünün yeniden test edilmesi.
- Sistem testi türleri: düzeltme (kurtarma) testi, güvenlik testi, dayanıklılık (stres) testi, yetenek (performans) testi.
- Onaylama testi + konfigürasyon (düzen) incelemesi = kabul muayenesi.
- Alfa testi: kullanıcıya benzer bir grup, yazılım evinin içinde dener. Beta testi: potansiyel gerçek kullanıcılar dener.
- Kara kutu testinin bulduğu hatalar: eksik veya hatalı işlevler, ara birim hataları, veri yapısı ve veritabanı erişim hataları, performans hataları, başlama ve bitirme hataları.
- Debug ile test farkı: test hatayı bulur, debug hatanın nedenini bulup düzeltir.
- Hata düzeyleri: ölümcül, kritik, büyük, orta, küçük, görünüm (kozmetik).

Detaylı Açıklamalar

Proje Duyuruları

- Gruplar en çok 5 kişidir. Beşinci kişi bulunamazsa 4 kişi de olabilir.
- Grup ve proje önerisi göndermeyenler bu hafta Cuma akşamına kadar gönderebilir. Sonradan grup değişikliği (örneğin beşinci kişi ekleme) kabul edilmez.
- Geç gönderenin proje notundan puan kırılır.
- Proje önerileri hocanın iki e-posta adresine de gönderilebilir. Hoca her gruba ayrı cevap verecektir. Konusu uygun olmayan gruptan konu değiştirmesi istenecektir.
- Seçilecek yazılımın kodu olmalıdır, çünkü birim testi istenecektir. Yazılımın alanı ve büyüklüğü serbesttir; önemli olan test ederken sıkıntı çıkarmamasıdır.
- 6. haftada gereksinim belgesi (SRS) istenecektir. Gereksinim belgesi için derste ayrı bölüm yoktur; Sistem Analizi ve Yazılım Mühendisliği notlarına bakın.
- Projede istenecekler:
  - Test planı: tarih takvimi, ekipte kimin hangi işi yaptığı.
  - Birim testi, sistem testi (araç serbest, araştırarak bulunacak) ve kullanıcı testi (gereksinim belgesindeki fonksiyonel özelliklere göre).
  - Kullanılan araçların tanımı, test verisi, test senaryoları.
  - Test sonuç raporu.
- Ayrıntılı kıstaslar sonraki haftalarda yazılı olarak verilecektir.

Yazılım İncelemesi Tekrarı

- İnceleme şu belgelerde yapılabilir: gereksinim spesifikasyonu, yüksek seviye tasarım, formal spesifikasyon, ayrıntılı tasarım, program.
- Program testi ise yalnızca çalıştırılabilen programlarda ve prototiplerde yapılabilir.
- Genel inceleme süreci üç adımdır: planlama ve hazırlık, inceleme ve toplantı, düzeltme ve tamamlama.
- Fagan incelemesi bu adımları altıya böler: planlama, genel gözden geçirme, kişisel kod inceleme, inceleme toplantısı, düzeltme, raporlama/tamamlama.
- Toplantıda kod yazanlar, inceleyenler, kayıt tutan kişi ve kalite grubundan biri bulunur.
- Etkili kod okuma hem yukarıdan aşağı hem aşağıdan yukarı yapılır.
  - Önce program en küçük parçalara (fonksiyon, prosedür) ayrılır ve her parçanın kendi içindeki doğruluğu incelenir.
  - Sonra parçaların programın bütünüyle uyumu incelenir.
- İnceleme ekibi: inceleme sorumlusu, inceleyiciler, kod yazanlar, kod okuyucu, kayıtçı.
- Teknik inceleme özet raporu şunları içerir: ne gözden geçirildi, kim gözden geçirdi, hangi hata ve eksikler bulundu, neler yapılmalı.

V Model ve Test Süreci Adımları

- V modelin sol kolu geliştirme adımları, sağ kolu test adımlarıdır.
  - Sol kol: müşteri gereksinimleri, ürün spesifikasyonu, yüksek düzey tasarım, alçak düzey tasarım, kodlama.

  - Sağ kol: birim testi, bileşen testi (varsa), bütünlük testi, sistem testi, kullanıcı kabul testi.
- Ayrıntılı şekilde bütünlük testinin üstüne regresyon testi, sistem testinin üstüne kabul testi (alfa, beta) eklenir.
- Şekil üç şeyi birlikte gösterir:
  - Test adımlarının sırası.
  - Her test adımının hangi geliştirme belgesini kullandığı. Bütünlük testi tasarım belgesini, sistem testi ürün/gereksinim spesifikasyonunu, kullanıcı testi fonksiyonel gereksinimleri kullanır.
  - Doğrulama ve geçerlemenin nerede yapıldığı.
- Waterfall modelinde test kodlamadan sonra başlar. Çevik (agile) modellerde test adımları geliştirme adımlarıyla birlikte yürür; kodun bitmesi beklenmez.

Test adımı	Ağırlıklı yapılan
Birim testi	Doğrulama (verification), saydam kutu
Bütünlük testi	Doğrulama (verification)
Sistem testi	Doğrulama ve geçerleme
İşlemsel kullanım / kabul	Geçerleme (validation)

- Geçerleme (validation): “Doğru ürünü oluşturduk mu?”
- Doğrulama (verification): “Ürünü doğru yolla oluşturduk mu?” Algoritma, veri yapısı ve izlenen yol doğru mu, buna bakılır; bunun için iç yapıya ihtiyaç vardır.

Birim Testi

- Yazılım tasarımının en küçük birimi olan modül üzerinde uygulanır.
- Ayrıntılı tasarıma dayanır. Önemli kontrol yolları test edilir. Saydam kutu testi uygulanır.
- Çok sayıda modül üzerinde paralel yürütülebilir.
- Test edilenler: modülün ara birimi, yerel veri yapıları, kontrol yapıları arasındaki ana yollar, modülün sınırları ve veri sınırları.
- Test ortamı:
  - Test sürücü, test edilen birime test verisini gönderir ve sonuçları toplar.
  - Test edilen birimin çağırdığı modüller henüz test edilmemişse yerlerine kukla modül (stub) konur.
- Modüller genelde bağımsız değildir. Bu nedenle test sürücüleri ve kuklalar yazılım içinde geliştirilir. Test programları da diğer yazılımlar gibi planlanır.

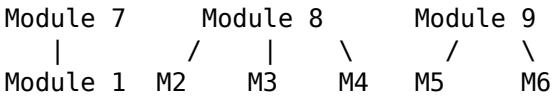
Ara Birim Testi

- Bütünlük testine hazırlıktır.
- Modüllerin birbirine veri göndermesi ve aralarındaki kontrol yapıları test senaryolarıyla denenir.
- Örnek: A, B, C modüllerinin birbirine gönderdiği veriler için test verisi hazırlanır.

Bütünlük (Bütünleme) Testi

- Modüller birbirine ara birimlerle (interface) bağlıdır.
- Birleştirmede çıkabilecek sorunlar:
  - Modüller arasında veri kaybı.
  - Modüllerin birbirini ters etkilemesi.
  - Alt fonksiyonlar birleşince beklenen ana fonksiyonun gerçekleşmemesi.
  - Her modülün kabul edilebilir hata toleransının birleşince büyümesi.
  - Ortak (genel) veri yapılarının sorun çıkarması.
- Bütün olarak sınamak zordur. Bu nedenle artırmalı sınama tercih edilir: modüller teker teker bağlanır.
- Slayttaki örnek ağaç (Şekil 5.3 ve 5.5):

```
          Module 10
  /          |          \
```



- Yukarıdan aşağı bütünleme (top-down): önce üst modül çalıştırılır, alttaki gruplar sırayla eklenir.
  - TD1: Module 10 ile Module 7, 8, 9.
  - Sonra Module 7 ile Module 1 (TD3), Module 8 ile Module 2, 3, 4 (TD2), Module 9 ile Module 5, 6 (TD4) eklenir.
  - Henüz eklenmemiş alt modüllerin yerine kukla modül kullanılır.
- Aşağıdan yukarı bütünleme (bottom-up): önce en alttaki modüller test edilip kümeler (cluster) halinde birleştirilir.
  - BU1: Module 1 ve Module 7.
  - BU2: Module 2, 3, 4 ve Module 8.
  - BU3: Module 5, 6 ve Module 9.
  - BU4: Module 7, 8, 9 ve Module 10.
  - Alt modülleri çağırmak için test sürücü kullanılır.
- Sandviç testi: yukarıdan aşağı ve aşağıdan yukarı yaklaşımın birlikte kullanılmasıdır.

Regresyon Testi

- Test edilmiş bir programda değişiklik veya ekleme yapılırsa bütünü yeniden test edilir.
- Uygulama ortamında değişiklik ve sabitleme yapıldıktan sonra da yeniden test yapılır.
- Amaç: değişiklikten sonra yazılımın yine hatasız çalıştığını göstermek.
- Bütünlük testi sırasında alt birimlerde değişiklik olursa bütünlük testi yeniden tamamlanır. Bu da regresyon testidir.

Sistem Testi

- Bilgisayar sistemi donanım ve yazılımdan oluşur. Yazılımı tek başına test etmek yetmez; bilgi sistemi içinde denetlemek gerekir.
- Soru: Yazılım işletim sistemi ve veritabanı sistemiyle ne kadar uyumlu çalışıyor?
- Amaç: sistemin bütün öğelerinin uygun biçimde bir araya geldiğini ve her birinin işlevini tam yaptığını göstermek.
- Sistem testi türleri:
  - Düzeltme (kurtarma) testi (recovery testing).
  - Güvenlik testi (security testing): sistemin dış müdahalelere ve bilgi hırsızlığına karşı korunduğunu kanıtlamak.
  - Dayanıklılık testi (stress testing): miktar, sıklık veya hacim bakımından anormal yükte sistemin dayanıklılığını ölçmek.
  - Yetenek/performans testi (performance testing): gerçek zamanlı ve gömülü sistemlerde işlem süresinin sisteme uygunluğunu test etmek.
- Performans testi her test basamağında uygulanabilir. Örneğin birim testinde modülün performansına bakılabilir.
- Performans testinde yapılanlar: performans seviyelerini belirlemek, belirli yük altında çalışma zamanı performansını ölçmek, sistem ayarlamaları (tuning).
- Performans testi şu sorulara cevap verir:
  - Sistem performans gereksinimlerini karşılıyor mu?
  - Normal şartlarda sistem nasıl davranıyor?
  - Trafik artışı işlem süresini ve işlevselliği nasıl etkiliyor?
  - Hangi kullanıcı sayısında performans sorunu başlıyor?
  - Performans düşüşü hangi bileşenden kaynaklanıyor?

Onaylama Testi, Alfa ve Beta Testi, Kabul

- Bütünleme sonunda yazılım bir paket haline gelmiş, ara birim hataları düzeltilmiş, sistem üzerinde çalışması değerlendirilmiştir.
- Onaylama testi, yazılımın müşteri ve kullanıcı beklentilerini karşılayıp karşılamadığına bakar.
  - Bütünleşmiş yazılım, gereksinimler ve kullanıcı belgeleri birlikte kullanılır.

  - "İstediğim gereksinimlerin hepsi var mı? Kullanıcı belgeleri beni doğru yönlendiriyor mu?"
- Konfigürasyon (düzen) incelemesi: kullanıcı belgeleri, tasarım belgeleri, kayıtlar ve test belgeleri birbiriyle uyumlu ve teslim edilebilir durumda mı?
- Onaylama testi ve konfigürasyon incelemesi birlikte kabul muayenesini oluşturur. Ardından yazılım teslim edilebilir.
- Alfa testi:
  - Sistem testinden sonra, onaylamadan önce yapılır.
  - Yazılımı geliştirenler değil, kullanıcıya benzer bir grup yazılım evinin içinde dener.
  - Grup gereksinim belgelerini, yardım dosyalarını ve menüleri izleyerek beklenen işlemlerin yapılıp yapılmadığına bakar.
  - Bu grupta dikkat ve takip yeteneği önemlidir.
- Beta testi:
  - Yazılım potansiyel gerçek kullanıcılara verilir.
  - Günlük kullanılan birçok yazılımın beta sürümü buna örnektir.
  - Önemli hata bildiren kullanıcılara ödül verildiği de olur.

Test Tipleri: Saydam, Kara ve Gri Kutu

- Kara kutu testi: fonksiyonel, performans ve dayanıklılık testleri dış spesifikasyona ve gereksinimlere dayanır. İç yapıyla ilgilenmez; "dış test" de denir.
- Saydam kutu testi: modül düzeyinde deyimler, dallar ve akış yolları test edilerek iç yapı incelenir; "iç yapı testi" de denir.
- Gri kutu testi: ikisinin karmasıdır. Örneğin test verisi saydam kutu bilgisiyle hazırlanır, test kara kutu biçiminde yapılır; ya da tersi.

Saydam Kutu Testi

- İşlemsel tasarımın kontrol yapısını kullanır. Sağladıkları:
  - Modüldeki bütün bağımsız yollar en az bir kez çalışır.
  - Bütün mantıksal kararların doğru ve yanlış durumları denenir.
  - Bütün döngüler kendi içinde ve sınırlarında denenir.
  - İç veri yapılarının geçerliliği güvenceye alınır.
- Temel yol testi (basis path testing):
  - İşlemsel tasarımın mantıksal karmaşıklığı ölçülür ve bu ölçüye göre bir temel yol grubu oluşturulur.
  - Adımlar: ayrıntılı tasarımdan veya kaynak koddan akış grafı çizilir; McCabe döngüsel karmaşıklık ölçüsü bulunur; bağımsız yolların temel grubu belirlenir; her yol için test verisi ve test programı hazırlanır; test uygulanır ve beklenen sonuçla karşılaştırılır.
  - Program, temel yol sayısı kadar çalıştırılır. Böylece her yol test edilir.
- Döngü testi (loop testing):
  - Temel yol analizine ek olarak yapılır.
  - Amaç: başlama hatalarını, indis hatalarını, artırma hatalarını ve döngü sınırı hatalarını bulmak.
  - Döngü türleri: basit döngü, iç içe (yuvalanmış) döngü, birbirine bağlı (ardışık) döngü, yapısal olmayan döngü.
  - Örnek sorular: Döngü sonsuz döngüye gidebilir mi? İç içe döngülerin indisleri birbirini bozuyor mu? Bir döngüden diğerine yanlış atlama var mı?

Kara Kutu Testi

- Yazılım ara birimi üzerinde yapılır. Bütünleme ve sistem testlerinde uygulanır.
- Sorular: Yazılım işlevlerini yapıyor mu? Girdileri kabul ediyor mu? Doğru çıktıları üretiyor mu?
- Fonksiyonel olmayan gereksinimler de sistem testinde kara kutu biçiminde denenebilir.
- Kara kutu test modeli (slayt):

```
Giriş sınama verisi (G)  <-- anormal davranışlara sebep olan girişler de eklenir
          |
       [Sistem]
```

```
          |
Çıkış sınama sonucu (Ç)  --> mevcut hataları ortaya çıkaran çıkışlar
```

- Yalnızca doğru girdiler değil, anormal davranışa yol açacak girdiler de bilerek gönderilir. Soru: Sistem tümden çöküyor mu, yoksa hata mesajı verip çalışmaya devam ediyor mu?
- Başlıca kara kutu yöntemleri: eş değerli bölümleme, sınır değer analizi, neden-sonuç grafiği (cause-effect graphing), veri onaylama testi. Derste ilk ikisi anlatıldı.

Eş Değerli Bölümleme

- Girdi alanı veri sınıflarına bölünür. Her sınıftan en az bir test verisi seçilir.
- Böylece her test belli bir sınıftaki hataları ortaya çıkarır. Daha az testle yetinilir.
- Geçerli girdiler aralıklara bölünür; geçersiz girdiler de ayrıca gönderilir.
- Örnek: yaş girdisi 0-99 arası.
  - Rastgele seçimde 1, 97, 87, 99 gibi değerler gelebilir; bunların çoğu aynı bölgeden olur.
  - Sınıflara bölünürse (0-15, 15-45, 45-80, 80-99) her sınıftan bir değer seçilir. Daha iyi bir test olur.
- Her test bir maliyettir. Optimum test sayısını yapay zekâ yöntemleriyle bulmak bir araştırma konusudur.
- Bir öğrencinin sorusuna cevap: amaç aralıklar oluşturup gereksiz test sayısını azaltmak, testi daha verimli yapmaktır.

Sınır Değer Analizi

- Hatalar genelde merkezde değil, kenarlarda (uç değerlerde) toplanır.
- Test verisi girdi ve çıktı alanlarının, veri yapılarının ve alt program parametrelerinin sınırlarından seçilir: en büyük, en küçük, en az beklenen değerler.
- Girdi alanında olduğu kadar çıktı alanında da uygulanır. Bunu destekleyen araçlar vardır.

Keşif Testi

- Ürün spesifikasyonu veya gereksinim belgesi olmadan yazılımı çalıştırıp kullanarak yapılan testtir.
- Yazılımın kendisi spesifikasyon gibi kullanılır. Dinamik kara kutu testi olarak da anılır.

Sürekli Güncellenen Yazılımlarda Test

- Bir öğrenci sordu: Sürekli güncellenen bir uygulamada (Spotify gibi) testler her sürümde baştan mı yapılır?
- Hocanın cevabı:
  - Testler her seferinde baştan yapılmaz. Önceki testler, test verileri ve test senaryoları arşivde saklanır; yeni eklemeler bunların üzerine konur.
  - Bu nedenle sürüm yönetimi (release management) son yıllarda önem kazanmıştır.
  - Sürüm yönetimi, konfigürasyon (düzen) yönetimi ile birlikte yürür. Konfigürasyon yönetimi hangi anda hangi değişikliğin yapıldığını ve yazılımın bütün elemanlarını (kod, gereksinim belgesi, tasarım belgesi, test belgeleri) izler.

Hata Giderme (Debugging)

- Test hatalı bölümleri bulur. Debug, bulunan hatanın nedenini bulup düzeltir.
- Önce hatanın hangi belirtilerle görüldüğü anlaşılır, sonra düzeltilir.
- Hata kaynakları: gereksinim analizi hataları, tasarım hataları, kod hataları.
- Gereksinim hataları, kullanıcının isteğini doğru anlatamaması ve yazılımcının onu doğru yönlendirememesinden doğar.
- Derste soru: Çevik yöntemlerde gereksinim hataları neden azaldı?
  - Öğrencilerin cevapları: gereksinim ile kod arasındaki süre kısa; iteratif ve küçük adımlarla ilerleniyor (özellikle XP).
  - Hocanın vurguladığı cevap: müşteri veya müşteri temsilcisi ekibin aktif bir parçasıdır. Ürünü yalnızca sonda değil, her aşamada görür.

- Spiral modelde her adımda kontrol ve risk analizi vardır; gereksinim hataları azdır ama süreç çok maliyetlidir.

Hata Düzeyleri

Düzey	Test	Teslim
Ölümcül	Testin devamını engeller; test durdurulur	Teslim edilmez
Kritik	Test devam eder (bazı kurumlarda durdurulur)	Düzeltilmeden teslim edilmez
Büyük	Test devam eder	Giderilmeden teslim zararlıdır
Orta	Test devam eder	Bu hatayla teslimde telafisi mümkün sorunlar çıkabilir
Küçük	Test devam eder	Teslim edilebilir; sonra düzeltilir
Görünüm (kozmetik)	Test devam eder	Teslim edilebilir; renk, font, büyüklük gibi hatalardır

- Yazılımın devamı hakkında karar hata düzeyine göre verilir.

Derste Sorulan Sorular

- Regresyon testi nedir? Bir öğrencinin cevabı: yeni sürümde yapılan düzeltmelerin gerçekten düzeltilip düzeltilmediğinin kontrolü. Hoca ekledi: geliştirme sırasında alt birimde değişiklik olunca bütünlük testinin tekrarlanması da regresyon testidir.
- Bir yazılımı hangi sırayla test ederiz?
  - Bir öğrenci test yönetimi adımlarını saydı: planlama, test tasarımı (senaryolar), testin yürütülmesi, hata raporlama.
  - Hoca: bu cevap "testi nasıl yönetirim" sorusuna uygundur (planla, test verisi ve senaryoları oluştur, uygula, test raporu ve test sonuç raporu yaz).
  - Sorulan ise test süreci adımlarıdır: birim, bütünlük, sistem, kullanıcı testi.
- Saydam kutu test tipleri nelerdir? Temel yol testi ve döngü testi. Temel yol testinde mantıksal karmaşıklık ölçülür ve bağımsız yolların her biri test edilir.
- Validation ve verification nedir? Yukarıdaki V model bölümüne bakın.

Diğer Duyurular

- Önümüzdeki hafta yazılım ölçümü, ardından test yönetimi konusu işlenecektir.
- Slaytların özetlenmiş hali birkaç hafta sonra ders panosuna konacaktır.
- 28 Ekim öğleden sonrası ve 29 Ekim resmî tatildir.

Ders sonrası notlar

- Yaş örneğinde hoca aralıkları 0-15, 15-45, 45-80, 80-99 diye söyledi. Eş değerli bölümlemede sınıflar çakışmamalıdır. Doğru bölümleme örneği: 0-14, 15-44, 45-79, 80-99. Ayrıca negatif sayılar ve 99'dan büyük değerler geçersiz sınıf olarak ayrıca denenmelidir.
- Sınır değer analizi ile eş değerli bölümleme birlikte kullanılır. Yukarıdaki örnekte sınır değer analizi 0, 14, 15, 44, 45, 79, 80, 99 ve geçersiz uçlar olarak -1, 100 değerlerini seçer.
- Hoca sistem testi türlerini sayarken "düzeltme testi"ni açıklamadı. Bu test İngilizce kaynaklarda recovery testing (kurtarma testi) olarak geçer: sistemin bir arızadan sonra kendini ne kadar sürede ve ne kadar doğru toparladığını ölçer.
- Hoca slaytta bazı adların yanlış kaldığını söyledi; V model şeklinde regresyon ve kabul testinin yeri bütünlük ve sistem testinin "yanında" değil "üstünde" olmalıdır.
- Bir öğrenci test süreci adımlarını sayarken kara kutu testini de bir adım olarak saydı. Kara kutu testi bir adım değil, test tipidir.

- Whisper altyazıda “bilim testi” geçer; doğrusu “birim testi”dir. “Arfa testi” de “alfa testi”dir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.